

01

From Königsberg to Knowledge Graphs

300 years of thinking about connections

Euler's Insight (1736)

The Seven Bridges of Königsberg

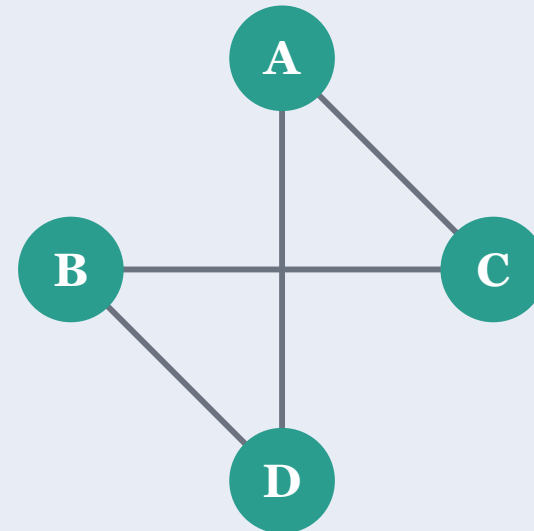
Can you walk through the city crossing each of its seven bridges exactly once?

Euler proved it was impossible — and in doing so, invented graph theory.

His key move: strip away the geography and focus only on the connections. Landmasses became nodes; bridges became edges.

This is exactly what a graph database does: it makes relationships the primary object of study, not the things being connected.

The Abstraction



Nodes = landmasses Edges = bridges

Abstraction

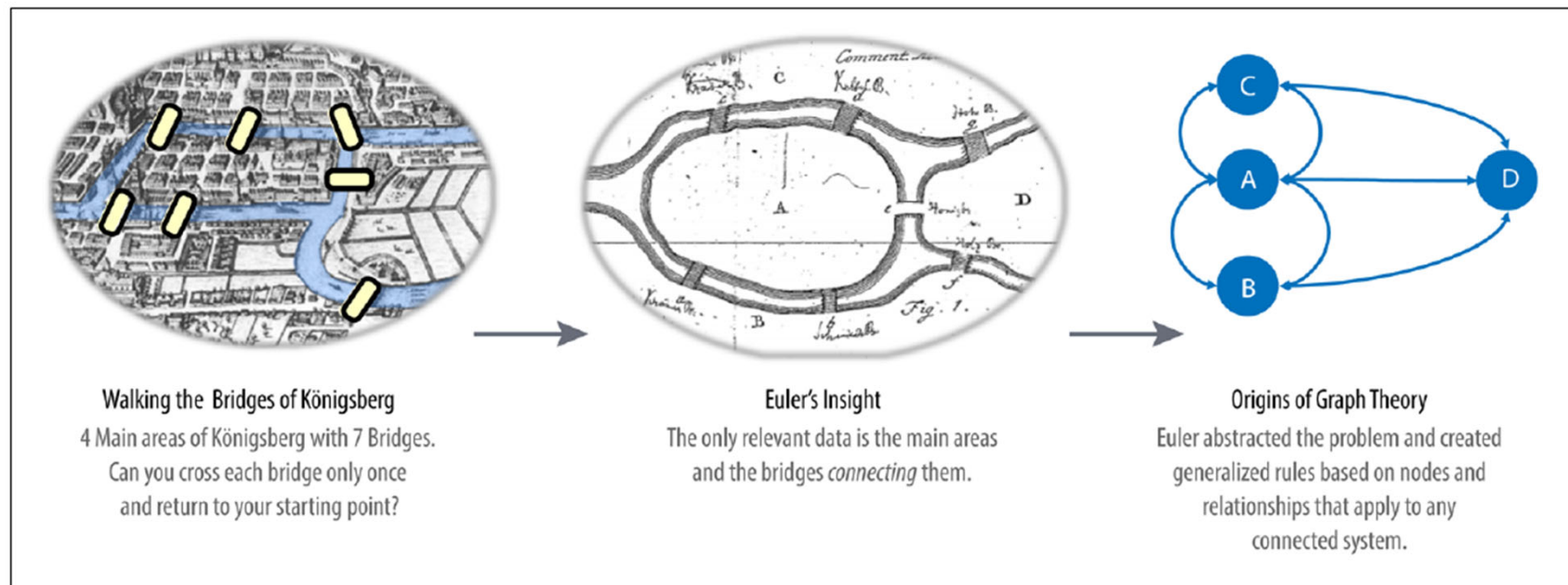
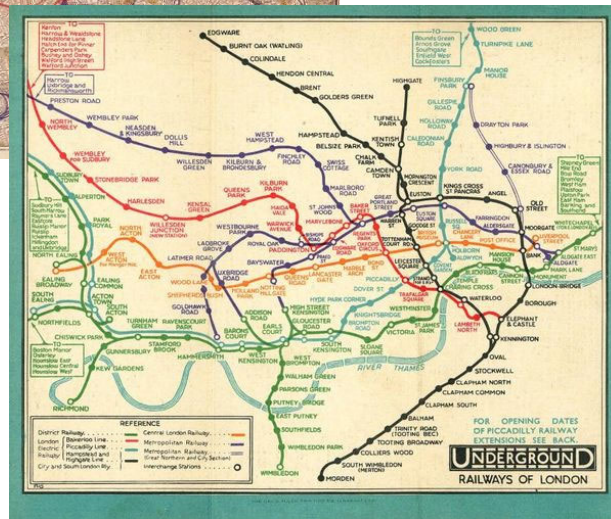
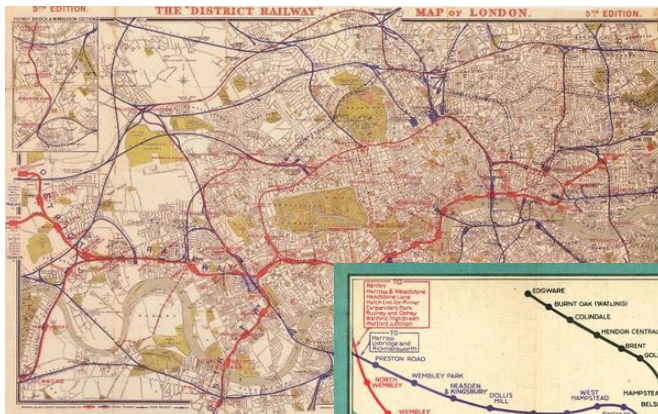


Figure 1-1. The origins of graph theory. The city of *Königsberg* included two large islands connected to each other and the two mainland portions of the city by seven bridges. The puzzle was to create a walk through the city, crossing each bridge once and only once.

Abstraction Reveals Connections



In a relational database,
data is stored in tables.
Relationships are defined in the schema.

**What if the schema was stored as
data?**

That's a graph database.

02

The Labeled Property Graph Model

The data model that underpins Neo4j, GQL, and most modern graph databases

Graph Terminology

Nodes (vertices)

The entities in your graph — people, products, documents, locations. Each node can have labels and properties.

Relationships (edges)

Connections between nodes. Always have a type and direction. E.g., Alice -[:FRIENDS_WITH]-> Bob.

Labels

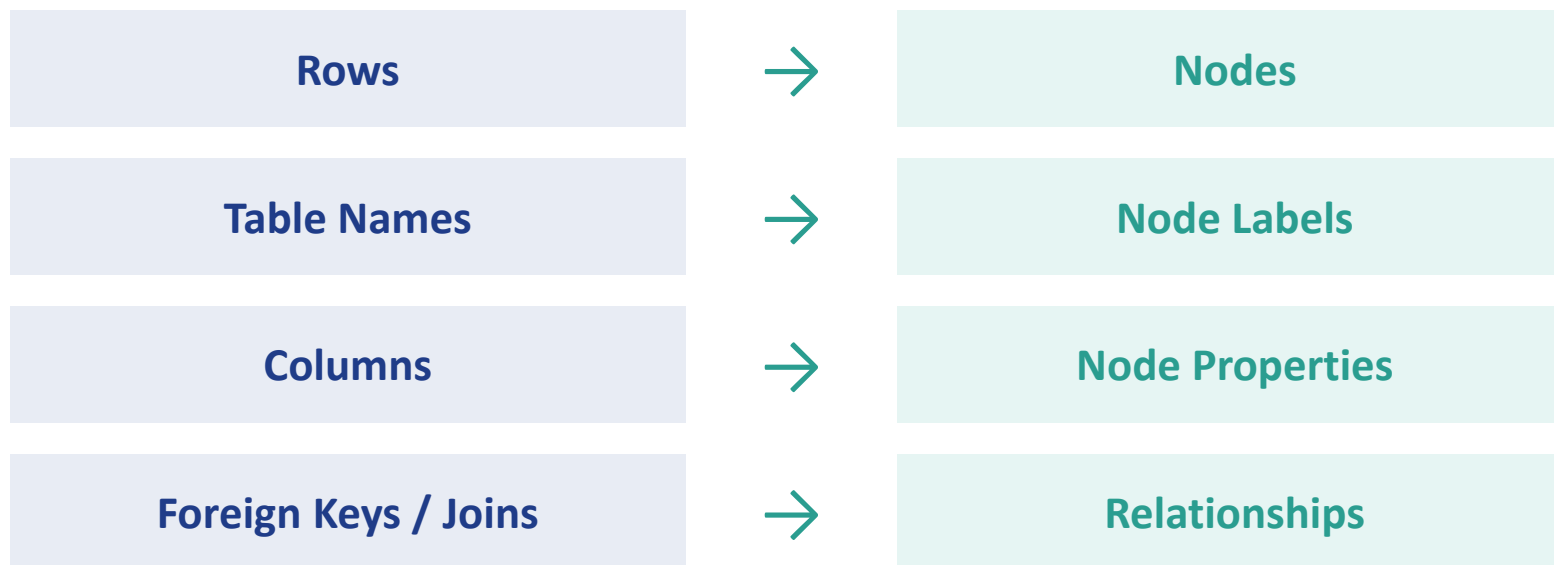
Group nodes into categories: Person, Movie, Department. A node can have multiple labels.

Properties

Key-value pairs on nodes or relationships: {name: 'Alice', age: 34, since: 2019}.

From Relational to Graph

Everything you know maps directly



Cat Cafe Example

A JOIN between Adopters and Cats → an ADOPTED relationship. A JOIN between Cats and Breeds → a TYPE_OF relationship. The schema becomes navigable data.

03

Neo4j and Cypher

The most popular graph database and its ASCII-art query language

Neo4j: The Graph Database Leader

Native Graph Storage

Purpose-built storage engine optimized for traversals. Relationships are stored as direct pointers — no join tables, no index lookups. Graph traversals scale at $O(1)$ per hop.

Neo4j 5.x (2025–26)

Multi-database support, 10x faster pattern matching, native parallelism, native vector search (cosine similarity), enhanced security with role-based access control.

ACID Compliant

Full transactional support with multi-database isolation. Enterprise-grade reliability for production workloads. Not just analytics — it's an operational database.

AuraDB (Cloud)

Fully managed cloud offering with serverless and dedicated tiers. Auto-scaling, automated backups, global availability. The Bedrock/RDS equivalent for graphs.

Cypher: Graph Query Language

ASCII art that queries graphs — declarative, readable, powerful

The Pattern Syntax

Node patterns:

```
(p:Person {name: 'Alice'})
```

Relationship patterns:

```
-[:FRIENDS_WITH]->
```

Combined:

```
(a:Person)-[:ACTED_IN]->(m:Movie)
```

Nodes use () parentheses.

Relationships use [] brackets.

Arrows show direction.

Core Clauses

MATCH	Find patterns in the graph
WHERE	Filter results by conditions
RETURN	Specify what to output
CREATE	Add new nodes/relationships
MERGE	Create if not exists, match if it does
SET	Update properties
DELETE	Remove nodes/relationships
WITH	Chain query parts (like a pipe)

Cypher in Action: Basic Queries

Find a person by name

```
MATCH (p:Person {name: "Tom Hanks"})  
RETURN p
```

Finds the node labeled Person with name property = 'Tom Hanks'

Find actors in a movie

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie  
  {title: "Toy Story"})  
RETURN p.name
```

Traverses ACTED_IN relationships to find all actors

Find friends of friends

```
MATCH (user:Person {name: "John"})  
  -[:FRIENDS_WITH]->(friend)  
  -[:FRIENDS_WITH]->(fof)  
WHERE user <> fof  
RETURN DISTINCT fof.name
```

Two-hop traversal — trivial in Cypher, complex multi-join in SQL

Cypher in Action: Creating Data

Create a node

```
CREATE (p:Person {name: "Emily Blunt",  
  born: 1983})  
RETURN p
```

Adds a new Person node with name and birth year properties. Returns the node so you can verify.

Create a relationship

```
MATCH (p:Person {name: "Emily Blunt"}),  
  (m:Movie {title: "A Quiet Place"})  
CREATE (p)-[:ACTED_IN {role: "Evelyn"}]->(m)
```

MATCH first to find existing nodes, then CREATE the connection. The relationship has its own properties.

MERGE (create if not exists)

```
MERGE (p:Person {name: "Emily Blunt"})  
  -[:ACTED_IN]->  
  (m:Movie {title: "A Quiet Place"})  
RETURN p, m
```

Atomic upsert — creates the entire pattern if absent, matches it if present. Prevents duplicates in idempotent loaders.

Cypher in Action: Real-World Patterns

Recommendation Engine

```
// Products purchased by customers who also bought Widget X
MATCH (p:Product {name: "Widget X"})<-[:PURCHASED]-(c:Customer)
    -[:PURCHASED]->(rec:Product)
WHERE p <> rec
WITH rec, count(*) AS frequency
RETURN rec.name, rec.price, frequency
ORDER BY frequency DESC LIMIT 5
```

Social Network: Find Shared Interests

```
// Friends-of-friends who like the same movies
MATCH (user:Person {name: "John"})-[:FRIENDS_WITH]->(friend)
    -[:FRIENDS_WITH]->(fof),
    (user)-[:LIKES]->(movie:Movie)<-[:LIKES]-(fof)
WHERE user <> fof
RETURN DISTINCT fof.name, movie.title
```

Graph vs. Relational: When Each Wins

Dimension	Relational	Graph
Data model	Tables, rows, columns	Nodes, relationships, properties
Schema	Rigid, predefined	Flexible, evolves with data
Relationships	Foreign keys + JOINS	First-class objects with properties
Multi-hop queries	Exponential JOINS (slow)	O(1) per hop (fast)
Best for	OLTP, reporting, structured data	Connected data, traversals, recommendations
Weakness	Complex JOINS at depth	Simple aggregations, bulk analytics
Scale pattern	Vertical (bigger machine)	Horizontal + traversal optimization

04

GQL: The New ISO Standard

The first new ISO database language since SQL in 1987

GQL: ISO/IEC 39075 (April 2024)

The first new ISO database language standard since SQL in 1987

GQL is to graph databases what SQL is to relational databases — a vendor-neutral standard that enables portability of queries, skills, and tooling across platforms.

Developed by

ISO/IEC JTC 1/SC 32 WG3 — the same committee that maintains SQL.

Contributors

Neo4j, Oracle, TigerGraph, Amazon, Google, Microsoft + academics.

Scope

Equivalent in size to SQL-92 — substantial, but still v1.0; vendor extensions permitted.

Relationship to Cypher

Same pattern syntax (ASCII art). Key change: GQL uses INSERT instead of CREATE. Neo4j supports both.

SQL/PGQ companion

SQL extension lets you call read-only graph queries from a SELECT. Bridges existing SQL infrastructure.

Adopters

Neo4j, Amazon Neptune, Microsoft Fabric, TigerGraph — all committed to GQL support.

05

Knowledge Graphs and GraphRAG

When vector search isn't enough — combining structure with similarity

Knowledge Graphs

Gartner: "Critical Enabler" with immediate impact on Generative AI (2024)

A knowledge graph is a structured representation of real-world entities and relationships. Nodes represent things (people, organizations, policies, medications). Relationships capture how they're connected (funds, regulates, treats, cites). Properties attach facts to both (amounts, dates, confidence scores). Unlike a flat document collection, a knowledge graph preserves the logical structure of how facts relate to each other.

Enterprise AI

Unify meanings across data silos. Enable AI agents to navigate data safely with semantic context.

Financial Services

Link companies, filings, transactions, and officers. Power compliance checks and investment analysis.

Healthcare

Connect patients, diagnoses, treatments, and outcomes. Cedars-Sinai built a 1.6M-edge Alzheimer's research graph.

Government

Map legislation, agencies, programs, and regulations. Enable cross-agency data integration.

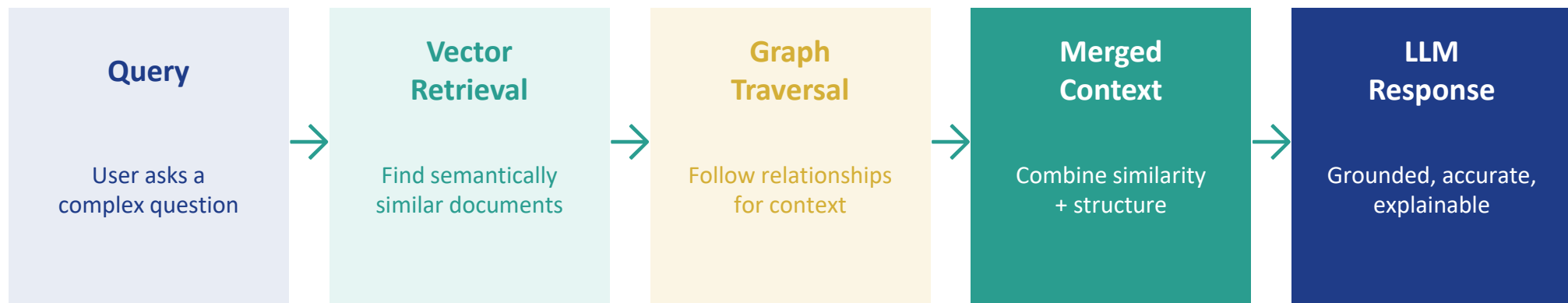
GraphRAG vs. Vector RAG

Dimension	Vector RAG	GraphRAG
Data structure	Flat embeddings in vector index	Entities & relationships as nodes/edges
Retrieval	Semantic similarity (top-k)	Graph traversal + local/global search
Multi-hop queries	Fails — each chunk is independent	Succeeds — traverses logical chains
Explainability	Hard to trace why a chunk was found	Traceable paths and source nodes
Context assembly	Top-k chunks by similarity score	Subgraph + community summaries
Cost	Low indexing, low query cost	100–1000x indexing cost, lower query cost
Best for	Simple factual lookups	Complex, connected, multi-source queries

Microsoft benchmarks: GraphRAG achieved 72–83% comprehensiveness on global queries where baseline vector RAG could not answer at all.

The Convergence: Graph + Vector

The standard architecture by 2027 — and it's already here in 2026



Neo4j now supports native vector search alongside graph traversals — combining both paradigms in a single system.

Amazon Neptune Analytics supports vector similarity search alongside graph queries.

PostgreSQL with pgvector + Apache AGE can do vector search AND graph queries — the Swiss Army knife.

The binding constraint lens: you don't choose between vector and graph — you use vector for "what's similar?" and graph for "how is it connected?"

06

Choosing the Right Tool

A decision framework — not a ranking

When to Use What

Is your data structured with clear schemas?	→ Relational (PostgreSQL, RDS)
Do you need semantic similarity search?	→ Vector (pgvector, Pinecone, Milvus)
Are relationships the core of your analysis?	→ Graph (Neo4j, Neptune)
Do you need multi-hop reasoning over connected data?	→ Graph + optional vector (GraphRAG)
Do you want one database for everything?	→ PostgreSQL (pgvector + Apache AGE)
Do you need RAG for an LLM application?	→ Vector for simple Q&A; GraphRAG for complex queries

These are not mutually exclusive. Modern architectures commonly combine two or three paradigms. The binding constraint is which relationship structure (tabular, semantic, or connected) best matches your dominant query pattern.

Graph Databases on AWS

Amazon Neptune

Fully managed graph database supporting both property graphs (openCypher/GQL) and RDF (SPARQL). ACID compliant, auto-scaling, multi-AZ. The primary AWS graph offering.

Neptune Analytics

Graph + vector in one service. Runs analytics and vector similarity search on graphs loaded from Neptune or S3. Supports graph algorithms (PageRank, community detection, shortest path) at scale.

Neptune Serverless

Auto-scales capacity based on workload. Pay-per-use pricing. Good for variable or unpredictable graph query patterns.

Key Takeaways: Week 13

- 1 Vector databases transform the question "what means the same thing?" into a geometric nearest-neighbor search. They are the infrastructure layer for RAG and LLM applications.
- 2 Graph databases make relationships first-class citizens. They excel where relational databases struggle: multi-hop traversals, connected data, and flexible schemas.
- 3 GQL (ISO 39075, April 2024) is the first new database language standard since SQL. It standardizes graph querying the way SQL standardized relational querying.
- 4 GraphRAG combines knowledge graphs with vector retrieval — solving the multi-hop reasoning, entity disambiguation, and global synthesis problems that vector-only RAG cannot.
- 5 The convergence trend: PostgreSQL (pgvector + Apache AGE), Neo4j (native vector search), and Neptune Analytics (graph + vector) show that these paradigms complement each other.

Questions?

Next week: Course review and final project

PPOL 5206 — Massive Data Fundamentals • Spring 2026